

QUESTION 1: AI-Powered Emergency Medical Drones in Dynamic Flood Zones

Scenario Summary: A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads/routes may suddenly become blocked, travel costs vary based on terrain and weather, and the drone has access to a straight-line distance heuristic, partial real-time updates, and variable movement costs. Route decisions must balance minimum delay and risk under extreme urgency.

i. Behavior, Strengths, and Weaknesses of Greedy Best-First Search Under Uncertainty

Operational Mechanics: Greedy Best-First Search (GBFS) operates by making purely locally optimal choices at each decision node. It expands the node that is estimated to be closest to the goal according to a heuristic function $h(n)$. In this disaster management scenario, $h(n)$ represents the Euclidean or straight-line distance from the drone's current geographic position n to the flood relief delivery site G . Mathematically, the node evaluation function simplifies to:

$$f(n) = h(n) = \sqrt{(x_G - x_n)^2 + (y_G - y_n)^2}$$

When navigating through the flooded region, GBFS prioritizes movement toward nodes that yield the maximum visual/spatial reduction in distance to G . It maintains a priority queue (OPEN list) ordered strictly by ascending order of $h(n)$.

Strengths in Dynamic Emergency Scenarios:

- **Low Computational Overhead & Time Complexity:** In emergency medical dispatch, every second counts. GBFS focuses aggressively on the goal, ignoring accumulated path costs $g(n)$. In non-pathological, sparsely blocked spaces, its search space is extremely narrow, allowing real-time trajectory generation within milliseconds.
- **Rapid Initial Path Generation:** When clear corridors exist, GBFS reaches the destination faster than unguided search strategies, making it ideal for immediate execution when rapid action is required.

Critical Weaknesses Under Environmental Uncertainty:

- **Incompleteness in Dynamic Obstacle Environments:** If floodwaters suddenly submerge a path or create dead-end pockets (e.g., surrounded by swollen rivers or collapsed structures), GBFS can easily fall into infinite loops or get trapped in local minima unless strict visited-state tracking is maintained.
- **Gross Suboptimality:** GBFS ignores accumulated movement costs $g(n)$. A path that moves straight towards the goal over severe high-risk storm terrain will be chosen over a slightly longer detour on calm, safe air corridors, potentially leading to drone damage or battery depletion.
- **Vulnerability to Dynamic Map Changes:** Because GBFS does not evaluate path history, encountering a dynamically blocked path forces the algorithm to backtrack wastefully or make drastic path alterations without considering prior energy expenditure.

ii. A* Search Mechanics: Balancing Path Cost and Heuristic Guidance

Mathematical Formulation: A* search overcomes the structural deficiencies of GBFS by evaluating nodes using a dual-component objective function:

$$f(n) = g(n) + h(n)$$

Where:

- **$g(n)$** is the actual cumulative cost incurred from the start position **S** to node **n** . In this scenario, **$g(n)$** integrates variable movement costs including weather severity (wind resistance), terrain elevation, risk factor of flying over open water, and battery energy consumption.
- **$h(n)$** is the estimated cost from node **n** to the target delivery site **G** (straight-line distance adjusted for average speed).

Balancing Act & Trade-off Resolution: A* balances backward cost (**$g(n)$**) and forward expectation (**$h(n)$**). If a route appears spatially direct (**$h(n)$** is low) but involves extreme weather turbulence or dangerous high-risk zones, **$g(n)$** increases rapidly. The total evaluation score **$f(n)$** will force the priority queue to prefer alternative paths that may be geographically longer but significantly safer and more energy-efficient.

iii. Impact of Heuristic Accuracy on Algorithm Performance

The performance and correctness of both GBFS and A* depend directly on the properties of the heuristic **$h(n)$** :

- **Admissibility ($h(n) \leq h^*(n)$):** A heuristic is admissible if it never overestimates the actual minimal cost to reach the goal. Straight-line Euclidean distance in a 2D/3D grid is strictly admissible because no physical path can be shorter than a straight line. Admissibility guarantees that A* will return an optimal path.
- **Consistency/Monotonicity ($h(n) \leq c(n, a, n') + h(n')$):** Ensures that the estimated cost never decreases along any valid path transition. Straight-line distance satisfies the triangle inequality and is therefore consistent. Under consistent heuristics, A* never needs to re-open nodes in the CLOSED set.
- **Impact of Heuristic Degradation in Dynamic Flood Zones:**
 - *Over-optimistic Heuristic:* Straight-line distance ignores physical obstacles (swollen rivers, temporary flight restrictions). While admissible, if obstacles force major detours, **$h(n)$** severely underestimates **$h^*(n)$** . A* degenerates towards Uniform Cost Search (expanding many nodes), while GBFS gets repeatedly pushed into dead-ends.
 - *Inaccurate / Non-admissible Heuristic:* If the drone overestimates travel costs based on false sensor data, A* loses its optimality guarantee and may yield unsafe, energy-expensive paths.

iv. Analysis of Dynamic Changes (Blocked Paths) on Search Performance

When flood conditions dynamically block a planned route during flight execution, static search algorithms suffer severe performance degradation:

The Dynamic Replanning Challenge

When a drone encounters an unexpectedly blocked corridor at location **$X_{current}$** , standard static search algorithms must drop the current tree and re-compute A* from **$X_{current}$** to **G** from scratch. In high-frequency changing environments, continuous full re-computation creates massive latency spikes, threatening drone stability.

In dynamic environments, static A* wastes computational resources by throwing away state space information previously explored. Dynamic graph search variants retain past cost bounds and propagate updates locally, preserving real-time readiness.

v. Recommendation & Comprehensive Justification

Recommended Algorithm: D* Lite (Dynamic A*) or Lifelong Planning A* (LPA*)

Detailed Justification:

- 1. **Real-Time Adaptation:** Standard A* is insufficient for dynamic flood zones due to high re-planning costs. D* Lite searches backward from the destination goal to the drone's current position. When new partial sensor updates detect a blocked path or weather shift, D* Lite incrementally updates only the affected nodes, executing replanning orders of magnitude faster than standard A*.
- 2. **Safety & Risk Management:** By maintaining a composite cost function $g(n)$ that incorporates risk factors (wind speed, precipitation, power requirements), D* Lite guarantees optimal or near-optimal paths while adapting instantly to hazards.
- 3. **Optimality Guarantees:** D* Lite maintains mathematical equivalence to A* replanning while achieving superior execution speeds, satisfying the critical constraints of time-critical medical delivery.

Criterion	Greedy Best-First Search	Standard A* Search	Recommended: D* Lite
Optimality	No (Sub-optimal)	Yes (Optimal)	Yes (Optimal)
Dynamic Replanning Cost	High (Restarts search)	Very High (Full re-computation)	Minimal (Incremental node update)
Safety / Risk Integration	Poor (Ignores path cost)	High (Includes cost factor $g(n)$)	Maximum (Dynamic risk adaptation)
Execution Speed in Urgency	Fast initial, slow on obstacles	Moderate initial, poor on updates	Optimal for real-time operations

QUESTION 2: Smart City Traffic Signal Optimization via Local Search & Metaheuristics

Scenario Summary: A smart city deploys an AI system to optimize traffic signal timings across hundreds of interconnected intersections. Objectives include minimizing total waiting time, fuel consumption, and traffic congestion. The state space is vast, traffic dynamics change continuously, and optimization algorithms risk getting trapped in local optima.

i. Optimization Problem Formulation & Inefficiency of Traditional Search

Formal Mathematical Formulation:

- **State Space (S):** A vector $S = \{t_{1,1}, t_{1,2}, \dots, t_{i,j}, \dots\}$ where $t_{i,j}$ represents the duration of green light phase j at signalized intersection i across N city intersections.
- **Constraints:** Minimum/maximum green phase bounds ($t_{\min} \leq t_{i,j} \leq t_{\max}$), cycle length constraints, and pedestrian crossing clearance times.
- **Objective / Cost Function (f(S)):**

$$\text{Minimize } f(S) = w_1 \cdot W_{\text{total}}(S) + w_2 \cdot F_{\text{total}}(S) + w_3 \cdot C_{\text{density}}(S)$$

Where W_{total} is cumulative vehicle delay (hours), F_{total} is estimated fuel burn (liters), C_{density} is queue length index, and w_1, w_2, w_3 are weighting factors.

Inefficiency of Traditional Systematic Search (BFS, DFS, A*):

- **Combinatorial State Explosion:** If there are 200 intersections, each with 4 phases and 10 possible discrete timing allocations, the total search space equals 10^{800} states. Systematic pathfinding algorithms (like A*) construct a search tree from an initial state to a goal state. Here, there is no explicit goal state; we seek an optimal configuration in an infinite continuous or high-dimensional discrete landscape.
- **Memory & Path Tracking Overhead:** Systematic search algorithms maintain path histories in memory (OPEN/CLOSED lists). In traffic timing optimization, the sequence of intermediate configurations does not matter—only the final configuration matrix matters. Storing path histories is computationally infeasible and irrelevant.

ii. Hill Climbing Mechanics and Fundamental Limitations

Algorithmic Execution: Hill Climbing is an iterative local search technique. It starts with an initial signal timing configuration S_0 and evaluates its cost $f(S_0)$. In each iteration, it evaluates immediate neighbors generated by tweaking single signal timings by small increments (e.g., ± 5 seconds). It transitions to the neighbor yielding the steepest reduction in $f(S)$.

```
Algorithm Simple-Hill-Climbing:
current_state = Initial_Traffic_Timing_Configuration()
loop do:
    neighbors = Generate_Neighboring_Timings(current_state)
    best_neighbor = Select_Neighbor_With_Min_Delay(neighbors)
    if Cost(best_neighbor) >= Cost(current_state):
        return current_state // Stuck at local peak/valley
```

```
current_state = best_neighbor
```

Primary Limitations in Traffic Domains:

- Hill Climbing makes purely greedy local decisions. It never accepts moves that temporarily increase traffic delay, causing it to freeze at sub-optimal configurations.

iii. Topological Hazards: Local Maxima, Plateaus, and Ridges in Smart City Grids

1. Local Maxima / Minima:

Real-World Interpretation: A sub-optimal traffic signal setup where adjusting any single signal's timing increases localized delay, even though globally re-coordinating multiple consecutive traffic signals along an arterial corridor would eliminate city-wide gridlock. Simple Hill Climbing terminates here prematurely.

2. Plateaus (Flat Local Surfaces):

Real-World Interpretation: A regional area in the state space where minor shifts in green light timings (e.g., adding 2 seconds to north-bound traffic during low-volume night hours) produce no measurable change in total fuel burn or delay. The algorithm performs a random walk without directional guidance, stalling optimization.

3. Ridges (Narrow Sequences of Local Maxima):

Real-World Interpretation: A complex inter-dependency where optimizing flow requires simultaneous, coordinated changes across multiple adjacent signals (e.g., green-wave synchronization along a boulevard). Independent single-variable adjustments lead to higher delays, causing the algorithm to ping-pong along the ridge without making forward progress.

iv. Overcoming Limitations via Simulated Annealing & Genetic Algorithms

Simulated Annealing (SA):

SA avoids getting trapped in local minima by allowing probabilistic "downhill" moves (moves that temporarily increase cost). The probability of accepting a worse state is governed by the Metropolis criterion:

$$P(\text{accept}) = \exp(-\Delta E / T)$$

Where $\Delta E = f(S_{\text{candidate}}) - f(S_{\text{current}})$ and T is the system Temperature. High initial temperatures allow wide exploration across poor configurations, escaping local minima. As T gradually cools according to an annealing schedule ($T_{k+1} = \alpha \cdot T_k$), SA transitions into standard Hill Climbing, converging to a near-global optimum.

Genetic Algorithms (GA):

GA relies on population-based parallel search inspired by biological evolution:

- **Chromosome Representation:** A binary or real-valued vector representing green-phase durations across all intersections.
- **Selection & Crossover:** High-performing configurations (low delay) are selected and combined via multi-point crossover, blending effective timing blocks from different intersections.
- **Mutation:** Random alterations inject new genetic material, preventing premature convergence across the entire urban network.

v. Recommendation & Comprehensive Justification

Recommended Hybrid Approach: Genetic Algorithm combined with Simulated Annealing (Memetic Algorithm) or Parallelized Genetic Algorithms.

Detailed Justification:

1. **Scalability across Large Networks:** GAs maintain a diverse population of candidate solutions simultaneously, enabling parallel exploration of large search spaces. This makes them significantly more effective than single-point search methods like Simulated Annealing when managing hundreds of intersections.
2. **Adaptability to Dynamic Traffic Flow:** When traffic patterns shift (e.g., rush hour or accidents), a GA does not need to restart from scratch. The current population provides a strong foundation that can adapt quickly through targeted mutation and crossover.
3. **Handling Multi-Objective Trade-offs:** Multi-Objective GAs (NSGA-II) generate Pareto-optimal fronts that effectively balance competing metrics like fuel efficiency, transit delays, and pedestrian crossing times.

QUESTION 3: Autonomous Mars Rover Navigation in Partially Observable Environments

Scenario Summary: An autonomous rover deployed on Mars must navigate safely to collect geological samples, avoid hazardous terrain (craters, boulders), operate with limited sensing capabilities, and make decisions without a complete prior map. The environment is dynamic and partially observable, requiring real-time online decision-making.

i. Necessity of Online Search vs. Offline Search Strategies

Offline Search Deficiencies: Offline search algorithms (such as classical A* or Dijkstra) assume complete knowledge of the environment state space prior to execution. They compute a complete, rigid plan from start to target offline. On Mars, offline planning fails because:

- High-resolution surface maps are unavailable prior to landing. Orbital images miss small hazards like deep sand traps or small boulders that can immobilize the rover.
- Dynamic events (dust storms, wheel slips, rock slides) alter the environment state after the initial plan is computed. An offline plan cannot handle unexpected obstacles during execution.

Online Search Mechanics: An online search agent interleaves computation and action. It performs local exploration using onboard sensors, computes a partial plan within its immediate field of view, executes actions, receives updated sensor feedback, updates its internal map, and replans iteratively.

ii. Challenges of Partial Observability and Dynamic Environments

The Martian environment presents severe sensing and decision-making challenges:

- **Partial Observability:** Onboard LiDAR and cameras have line-of-sight and range limitations. Shadows, dust storms, and terrain occlusions prevent the rover from observing full surface geometry beyond its immediate vicinity. The agent must maintain a probabilistic belief state over possible world configurations.
- **Dynamic & Non-Deterministic Terrain:** Loose Martian regolith creates non-deterministic movement: executing a "move forward 1 meter" command may yield 0.7 meters of movement due to wheel slippage. Environmental shifts like dust accumulation can also reduce solar panel output, introducing energy constraints during navigation.

iii. Internal Model Construction and Dynamic Belief Updating

To make safe decisions despite incomplete information, the rover maintains a dual-layer internal model:

1. Occupancy Grid Mapping & Hazard Elevation Maps:

The rover divides the physical surface into a 2D/3D grid. Each cell $c_{i,j}$ maintains a probability score $P(\text{Occupied} \mid z_{1:t})$ representing hazard likelihood, updated via Bayesian filtering using sensor measurements z_t :

$$P(c_{i,j} \mid z_{1:t}) = \alpha \cdot P(z_t \mid c_{i,j}) \cdot P(c_{i,j} \mid z_{1:t-1})$$

2. SLAM (Simultaneous Localization and Mapping):

Because wheel slippage introduces position uncertainty, the rover uses Extended Kalman Filters (EKF) or Particle Filters (FastSLAM) to simultaneously estimate its true position $\mathbf{x}_t$ and refine its environmental map $\mathbf{m}$.

iv. Comprehensive Comparison: Online Search vs. Uninformed & Informed Search

Dimension	Uninformed Search (BFS/DFS)	Informed Search (A*/Greedy)	Online Search (LRTA* / D* Lite)
Map Requirement	Requires full global graph	Requires full global graph	Operates on local/partial dynamic map
Execution Timing	Plans full path before moving	Plans full path before moving	Interleaves local planning & action
Handling Uncertainty	Fails completely on errors	Fails completely on errors	Continuously adapts via sensor feedback
Memory Overhead	Exponential / High	High (stores full search space)	Low (stores local horizon + visited notes)
Rover Safety Suitability	Unusable	Unusable in unknown terrain	Highly Suitable

v. Recommendation & Comprehensive Justification

Recommended Strategy: Learning Real-Time A* (LRTA*) integrated with POMDPs (Partially Observable Markov Decision Processes) and Local Hazard Avoidance.

Detailed Justification:

- 1. Guaranteed Completeness and Avoidance of Infinite Loops:** LRTA* updates its local heuristic estimates $H(s)$ dynamically as it explores. If a path leads to a dead end or sand trap, the estimated cost of that state increases, naturally driving the rover to explore safer alternate routes.
- 2. Adaptability under Sensor Noise:** Combining LRTA* with EKF-SLAM and POMDP formulation allows the rover to choose paths that balance goal progression with uncertainty reduction (e.g., navigating near identifiable landmarks to recalibrate its localization).
- 3. Mission-Critical Safety:** Offline algorithms can execute dangerous movements into unobserved hazards. The proposed online architecture enforces local safety verification before committing to movement, protecting the rover while maintaining mission progression.

QUESTION 4: Automated University Exam Timetabling via Constraint Satisfaction Problems

Scenario Summary: A university requires an automated AI scheduling system to generate exam timetables. The system must accommodate course overlap restrictions, limited exam halls, sequential course dependencies, faculty availability, student accommodations, and expanding course loads.

i. CSP Formalization: Variables, Domains, and Constraints

A Constraint Satisfaction Problem is formally defined as a triple (V, D, C) :

1. Variables (V):

Let $V = \{E_1, E_2, \dots, E_n\}$ represent the set of all course examinations that require scheduling during the exam period.

2. Domains (D):

Each variable E_i is associated with a domain D_i of available time slots and room allocations: $D_i = \{(t, r) \mid t \in T, r \in R\}$ where $T = \{Slot_1, Slot_2, \dots, Slot_m\}$ and $R = \{Hall_1, Hall_2, \dots, Hall_k\}$.

3. Hard Constraints (Must be strictly satisfied):

- **Student Overlap Constraint (Binary Constraint):** If Exam E_i and Exam E_j share registered students, they cannot share the same timeslot:

$$time(E_i) \neq time(E_j) \quad \forall (E_i, E_j) \text{ with common students}$$

- **Hall Capacity Constraint (Unary/Global Constraint):** The combined student enrollment of exams assigned to room r at time t must not exceed the hall's maximum capacity $Cap(r)$.
- **Prerequisite Sequence Constraint (Directional Constraint):** For dependent courses:

$$time(E_{prereq}) < time(E_{advanced})$$

- **Faculty Availability Constraint:** Exam E_i must be scheduled when assigned proctors are available.

4. Soft Constraints (Preferences to be maximized):

- Preventing students from having multiple exams on the same calendar day.
- Distributing heavy exams evenly across the examination window.

ii. Backtracking Search Mechanics in Timetabling

Backtracking search is a depth-first search strategy tailored for CSPs. It assigns values to one variable at a time and backtracks when a variable has no valid values remaining.

Algorithmic Execution Trace:

1. Select an unassigned examination E_i .

2. Iterate through candidate domain values $\mathbf{v} = (t, r) \in D_i$.
3. Check if assigning $E_i = \mathbf{v}$ violates any hard constraints against previously assigned exams.
4. If consistent, tentatively assign $E_i = \mathbf{v}$ and recursively proceed to the next exam.
5. If a constraint violation occurs and no valid assignment exists in D_i , backtrack to the previous assigned exam and try a different value.

iii. Efficiency Enhancements via Constraint Propagation & Heuristics

Standard backtracking is inefficient for large scheduling problems because it detects failures only after reaching deep dead ends. Constraint propagation identifies inconsistencies early, shrinking search spaces dramatically.

1. Forward Checking (FC):

Whenever variable $\mathbf{X}$ is assigned a value, Forward Checking inspects all unassigned neighbor variables $\mathbf{Y}$ connected by constraints. It removes any values from $D(\mathbf{Y})$ that conflict with $\mathbf{X}$. If any domain becomes empty, the algorithm backtracks immediately, avoiding dead-end branches.

2. Arc Consistency (AC-3 Algorithm):

AC-3 maintains consistency across all variable pairs. An arc $(\mathbf{X} \rightarrow \mathbf{Y})$ is consistent if for every value in $D(\mathbf{X})$, there is at least one valid value in $D(\mathbf{Y})$. If removing a value from $D(\mathbf{Y})$ causes another arc to become inconsistent, AC-3 re-checks affected variable domains recursively, pruning infeasible options early.

3. Variable & Value Ordering Heuristics:

- **Minimum Remaining Values (MRV):** Chooses the unassigned exam with the fewest legal remaining time/room slots. This prioritizes tightly constrained variables, failing early if no solution exists.
- **Degree Heuristic:** Breaks MRV ties by selecting the variable involved in the largest number of constraints with other unassigned variables (e.g., large multi-department core courses).
- **Least Constraining Value (LCV):** Prefers values that leave the maximum flexibility for remaining unassigned variables.

iv. Real-World Challenges and Recommendation for Scalability

Real-World Complexity Challenges: Large universities manage thousands of students, hundreds of courses, special accessibility requirements, and last-minute room adjustments. Exact deterministic CSP techniques can suffer from exponential time complexity in large, tightly constrained scenarios.

Recommended Strategy: Hybrid Backtracking with AC-3 / MRV / LCV combined with Genetic Local Search (Memetic Hybrid CSP).

Justification:

- **Phase 1 (Hard Constraint Resolution):** Arc Consistency (AC-3) combined with MRV-based Backtracking quickly identifies valid schedules that satisfy all hard constraints.
- **Phase 2 (Soft Constraint Optimization):** A local search metaheuristic (such as Tabu Search or Simulated Annealing) refines the schedule to maximize student preferences, spacing exams evenly without violating hard constraints.

QUESTION 5: Strategic Real-Time Game AI: Minimax, Alpha-Beta Pruning & Heuristic Design

Scenario Summary: A gaming company is developing an AI opponent for a real-time strategy (RTS) game. The AI must compete against human players under strict time limits per turn/frame, evaluating complex game states with massive action spaces and deep decision trees.

i. Problem Modeling using the Minimax Framework

Theoretical Foundation: Strategic adversarial games are modeled as deterministic, fully observable, two-player zero-sum games. The AI player (MAX) aims to maximize its minimum payload, while the opponent (MIN) works to minimize MAX's payoff.

Recursive Minimax Equation:

$$V(s) = \begin{cases} \text{Utility}(s) & \text{if Terminal-Test}(s) \\ \max_{a \in \text{Act}(s)} V(\text{Result}(s, a)) & \text{if Player}(s) = \text{MAX} \\ \min_{a \in \text{Act}(s)} V(\text{Result}(s, a)) & \text{if Player}(s) = \text{MIN} \end{cases}$$

ii. Computational Challenges of Large Decision Trees in RTS Games

The time complexity of standard unpruned Minimax is $O(b^d)$, where b is the branching factor (number of legal moves per state) and d is the search depth.

- In games like Chess, $b \approx 35$, $d \approx 80$.
- In Real-Time Strategy (RTS) games, multiple units act simultaneously. Selecting target positions for 20 individual units yields branching factors exceeding $b \approx 10^{30}$.
- Evaluating even 3 steps ahead requires searching $(10^{30})^3 = 10^{90}$ states—far exceeding the memory and processing limits of modern hardware during real-time gameplay (often restricted to 16–33 milliseconds per frame).

iii. Alpha-Beta Pruning Mechanics and Performance Optimization

Alpha-Beta pruning removes subtrees from the evaluation pipeline without affecting the final minimax decision, reducing the effective search space.

Alpha and Beta Definitions:

- α : The best (highest-value) choice found so far for MAX along the path. Initialized to $-\infty$.
- β : The best (lowest-value) choice found so far for MIN along the path. Initialized to $+\infty$.

Pruning Conditions:

At a MIN node, if a child evaluates to a value $v \leq \alpha$, MAX will never choose the parent node leading to this state. Search on remaining siblings is pruned immediately (α -cutoff). Similarly, at a MAX node, if $v \geq \beta$, MIN will avoid this branch (β -cutoff).

```
Function Alpha-Beta(node, depth, alpha, beta, IsMax):  
    if depth == 0 or Terminal(node): return Evaluate(node)
```

```
if IsMax:
    v = -infinity
    for child in node.children():
        v = max(v, Alpha-Beta(child, depth-1, alpha, beta, False))
        alpha = max(alpha, v)
        if beta <= alpha: break // Beta Cutoff
    return v
else:
    v = +infinity
    for child in node.children():
        v = min(v, Alpha-Beta(child, depth-1, alpha, beta, True))
        beta = min(beta, v)
        if beta <= alpha: break // Alpha Cutoff
    return v
```

Pruning Efficiency: Under optimal move ordering (evaluating best moves first), time complexity improves from $O(b^d)$ to $O(b^{d/2})$. This effectively doubles the searchable depth within the same compute budget.

iv. Comprehensive Design of an RTS Evaluation Function

When search depth is limited by time constraints, non-terminal leaves are evaluated using a weighted heuristic evaluation function:

$$Eval(S) = w_1 \cdot Material(S) + w_2 \cdot Positional(S) + w_3 \cdot Resources(S) + w_4 \cdot Threat(S)$$

Factor	Sub-Metrics Included	Strategic Rationale
Material Advantage ($w_1 = 0.35$)	Army unit counts, structural health, force composition balance.	Reflects direct military strength and combat power.
Positional Control ($w_2 = 0.25$)	High-ground occupancy, choke point control, visibility.	High ground provides tactical defense and attack bonuses.
Economic Infrastructure ($w_3 = 0.25$)	Worker unit counts, resource gathering rates, base expansion count.	Sustains long-term army production and technological progression.
Threat Assessment ($w_4 = 0.15$)	Proximity of enemy strike forces to vulnerable expansions.	Drives defensive reaction before base damage occurs.

v. Recommended Strategy for Real-Time Execution

Recommended Architecture: Iterative Deepening Minimax + Transposition Tables + Move Ordering + Monte Carlo Tree Search (MCTS) / Abstraction Layer.

Implementation Components:

- 1. Iterative Deepening Search (IDS):** The AI runs searching at depth 1, then depth 2, depth 3, etc. If the strict real-time budget (e.g., 30ms) expires, the algorithm aborts the current depth and immediately returns the best move from the last fully completed search depth.
- 2. Transposition Tables (Zobrist Hashing):** Caches results of previously evaluated game states. In large games, different move sequences often reach identical states; caching prevents redundant searches.

3. **Move Ordering via Killer Heuristics & History Tables:** Evaluates likely strong moves first (captures, tactical advances), maximizing early Alpha-Beta cutoffs.
4. **Macro-Action Abstraction:** Groups individual unit commands into high-level tactical commands (e.g., "Flank Left Base" instead of individual unit moves), drastically reducing the branching factor ***b*** to a manageable level.